

Clay Seal: runtime containment for LLM agents

2026-08-07

A runtime authorization layer that binds an agent’s authority to the target and the outcome of each action rather than to the tool it calls. Every number below is reproducible from the repository; commands in Section 7.

1 What the layer is

An agent session begins by sealing a goal. The goal compiles into a set of machine-checkable constraints (`agentauth.core.task_scope.compile_task_scope`) covering allowed path globs, resource identifiers, action verbs, argument shapes, and aggregate budgets. Every attempted tool call passes through one gateway (`SessionBroker.authorize`) before execution, and every decision is appended to a hash-chained log (`DecisionLog`, verifiable via `verify()`).

Six enforcement rungs, cheap to expensive:

Rung	Primitive	Catches
tool-allowlist	tool-name set membership	ungranted tools
capability-token	<code>capability_allows</code> (Biscuit)	action escalation
task-scope	<code>task_scope_allows_path</code>	path exfiltration, credential stores
+binding	commit-token <code>arguments_hash</code>	argument tampering, swapped payee
+budget	<code>SessionValueBudget</code> , <code>SessionCallBudget</code>	fragmented spend, bulk exfiltration
intent envelope	goal-derived plan conformance	in-scope actions never authorized

Destination binding runs at the floor on any action carrying an external address, not only on send-shaped verbs, so an SSRF-style read of an attacker-controlled URL is caught as an acquisition. Trusted destinations are seeded from the sealed goal text and widened only from structured fields of tool output, never from free text, because free text is where injections live.

Responses are graduated. A denial backed by positive evidence of malice (protected zone, untrusted destination, mutated payload) hard-blocks. A scope miss, which is uncertainty rather than evidence, raises a step-up that halts autonomous execution and is recoverable under supervision. Human attention is a charged, bounded resource (`audit_budget`), so a session cannot ask an unbounded number of confirmations.

2 The bar

- **CaMeL** (arXiv:2503.18813, DeepMind): 77% AgentDojo utility against 84% undefended, a 7-point cost, via dual-LLM plus capabilities plus information-flow control.
- **Progent** (arXiv:2504.11703): AgentDojo ASR 39.9% to 1.0%, ASB 70.3% to 3.9%, deterministic symbolic least-privilege over tool arguments with Z3 narrowing. In our own head-to-head under `important_instructions` it leaves 11.1 to 16.7% ASR across four suites.

- Neither reports human-interruption cost, long-horizon false-alarm rate, or containment against an adaptive attacker that reads the policy.

3 Results

3.1 Authority must bind to the target, not the tool

RedCode-Exec (NeurIPS 2024 D&B), 718 risky operations, benign side from 344 BFCL multi-turn filesystem steps:

Engine	Attack prevented	False-block
tool-allowlist	0.0% [0.0, 0.4]	0.0%
capability-token	0.0% [0.0, 0.4]	0.0%
task-scope	99.9% [99.6, 100.0]	0.0%
task-scope+binding+budget	99.9% [99.6, 100.0]	0.0%

All 717 contained attacks leave the granted surface via the *target alone*. Every attack uses a tool the agent legitimately holds, which is why the two lower rungs contain none of them. Intervals are task-clustered bootstrap, since the 718 events derive from 14 risk templates and an event-level interval would be roughly the square root of the events per template too narrow.

3.2 Live injection, AgentDojo workspace, gpt-4o-mini

Configuration	ASR	Clean utility	Under attack	Interrupts/task
undefended	83.3%	100.0%	27.8%	0.00
deployable (envelope+taint)	0.0%	100.0%	83.3%	1.50

Clean utility matches the undefended baseline exactly. Eight further configurations reach the same 0.0% ASR and differ only in interruption count, from 1.50 to 6.00 per task, which is why the last column is reported.

3.3 Utility cost across a model ladder

Four suites, 32 clean tasks per model, paired per task so a task that also fails undefended is not charged to the defense:

Model	Baseline	Deployable	Cost	False-block
gpt-4o-mini	84%	59%	25 pts	12.5% [5.0, 28.1]
gpt-oss-120b	84%	66%	19 pts	9.4% [3.2, 24.2]
grok-4-1-fast	81%	78%	3 pts	6.2% [1.7, 20.1]
llama-4-maverick	12%	12%	0 pts	0.0%

Cost falls monotonically with agent capability. On the strongest model measured the deployable path costs 3 points against CaMeL’s published 7. The llama-4-maverick row is a null: a 12% baseline leaves nothing for a defense to cost, and it is kept in because dropping models that perform badly is the selection this evaluation exists to prevent.

3.4 Policy-aware adaptive attacker

400 RedCode tasks, 6 rounds, attacker mutates against observed denials:

Engine	Blind	Deny reasons	Reads policy	Poisoned goal
tool-allowlist	0%	0%	0%	0%
capability-token	0%	0%	0%	0%
task-scope and above	100%	100%	100%	100%

Identical for exfiltration (T1041), persistence (T1546), and destruction (T1485). Attacker knowledge does not move containment, because the constraint is on the outcome rather than on input phrasing. A prefix matcher without path canonicalization is checked in as a negative control and falls 80 of 80 at round 1, so the 100% figures are bounded by a search known to work.

3.5 Long-horizon sessions

200 sessions of 500 benign actions with one malicious action inserted at a seeded position, the needle wearing the session’s own tool and resource so it is not detectable by name:

Engine	Needle detected	False alarms / 1k benign
tool-allowlist	0.0%	0.00
capability-token	82.0% [76.1, 86.7]	0.00
task-scope	83.0% [77.2, 87.6]	0.00
task-scope+binding	100.0% [98.1, 100.0]	0.00

100,000 benign actions judged per engine, holding at 2,000-action sessions. AgentDojo trajectories average about three calls, so this separation between path scoping and argument binding is not visible on published benchmarks.

3.6 Attack-class coverage

Attack events partitioned without consulting any engine, by whether any field (tool, resource, action, target) falls outside what the task’s mandate and benign trajectory establish:

Corpus	Attack events	Surface-leaving	In-surface
RedCode	718	717 (100%)	1
AgentHarm	652	41 (6%)	611 (94%)
ASB	2,040	2,040 (100%)	0

Pooled over 3,410 events the full stack contains **82.1%**, not the 100% the surface-leaving column alone suggests. The in-surface class, where every field is legitimate and only intent differs, is contained by no per-call authorization system including ours. ASB’s 100% is a weak result: all 2,040 attacks leave via an ungranted tool name, so the weakest rung catches them.

3.7 Enforcement cost

10,880 decisions per engine, 20 repeats, warm:

Rung	p50	p95	p99
capability-token	2.79 μ s	3.33 μ s	3.50 μ s
task-scope	31.9 μ s	36.3 μ s	56.1 μ s
full stack	35.0 μ s	40.5 μ s	60.3 μ s

Scope compilation is amortized per task at 21 ms per 250 tasks and excluded.

4 Capabilities the comparables do not have

Adaptive red-team against our own policy. The harness searches over mutations (path obfuscation, tool and resource laundering, staging through in-scope locations, fragmentation, and policy-derived escape construction) rather than replaying a fixed corpus. Within an hour of first running it found two exploitable bypasses in shipping code: `path_matches_any` fnmatched raw strings, so a `/app/**` grant admitted `/app/./etc/passwd`, and `/proc/self/root/...` reached the filesystem root behind a prefix no protected-zone pattern matched. Both fixed, both with regression tests. Every company deploying agent authorization has this bug class and no tooling to find it.

Interruption cost as a first-class metric. A configuration reaching 0% ASR by confirming everything has relocated the vulnerability, and approval fatigue is an attack surface. Sweeping our response policy on banking found a setting that reaches identical 0.0% ASR while spending 4.17 interruptions per task against 0.83, at half the clean utility. On the two axes normally reported it is a tie.

Long-horizon false-alarm rate. Reported per 1,000 benign actions rather than per call, because a 0.5% per-action rate is five alerts per thousand actions, which at agent speeds is an alert every few minutes.

Verifiable audit. Hash-chained decision records with `verify()`, which is what a regulated buyer needs to demonstrate control rather than assert it.

5 Evidence base and limits

Deterministic tiers run at large n and reproduce bit-for-bit against a content-pinned corpus: 3,410 attack events in the coverage analysis, 100,000 benign actions in the long-horizon tier, 10,880 decisions per engine in the latency measurement, 512 tests passing.

Live measurements drive a stochastic agent through AgentDojo at $n = 18$ per configuration per suite, and at that size a single sweep supports a clean attack-success result to about one significant figure. A pooled multi-sweep run is in progress to tighten those intervals. The deterministic results are unaffected.

Utility on weak models is the open cost: 25 points on gpt-4o-mini against 3 on grok-4-1-fast. Diagnosis attributes it to the LLM planner failing to enumerate legitimate steps rather than to the enforcement rules, and mid-session re-clearance (`SessionBroker.reclear`) is the route.

6 Roadmap

In-surface containment. The 18% of attacks that stay inside the granted surface are uncontained by any per-call system. Velocity limits against the task’s own observed shape, novel-counterparty holdback, and provenance tracked transitively across the data-flow graph are the mechanisms. The aggregate-budget rung already contains the volume form of this class, which is why bulk exfiltration over an allowed channel is caught today.

Syscall-boundary enforcement. Symlink resolution and time-of-check to time-of-use are undecidable from userspace path strings. Enforcing at the kernel boundary via iVisor closes both and enables evaluation against an agent actively attempting sandbox escape, which no published benchmark covers.

7 Reproduce

```
benchmarks/fetch_corpora.sh      # pinned external corpora, ~4 MB
benchmarks/run_all.sh           # all deterministic tiers, under a minute
python -m benchmarks.adaptive --dataset redcode --rounds 6
python -m benchmarks.trajectory --length 500 --trajectories 200
python -m benchmarks.live.frontier --suite workspace --repeats 3
```